MORPHEUS Manual

Algorithmic model generation and semiexperimental refinement in ORACLE

ORACLE Development Notes

July 2, 2026

Abstract

`MORPHEUS` is the `ORACLE` program for semiexperimental equilibrium-structure refinement from rotational constants of isotopologues, vibrational and electronic corrections, and optional reference structural information. The program fits Cartesian molecular structures while automatically constructing a statistically defined active model from topology, symmetry, isotopic coverage, constraints, predicates and parameter classes. This manual describes the recommended command-line and graphical workflows, the input formats, the constraint language, the numerical options and the standard output files.

Contents

1 Purpose and Design Principles

`MORPHEUS` refines an equilibrium geometry by minimizing the difference between corrected experimental rotational information and the values calculated from a current Cartesian structure. The recommended observable is the principal moment of inertia, because it avoids the reciprocal nonlinearity of rotational constants and gives a more stable least-squares problem. Rotational constants are still reported at the end of every run for direct spectroscopic inspection.

The program is designed around three rules.

1. The active model must start from Cartesian atoms and perceived topology, not from a user-designed Z-matrix.
2. The fitted space must be non-redundant, symmetry-adapted and statistically well defined.
3. Constraints and parameter classes should be generated or expressed in chemically transparent primitive coordinates, even when the active least-squares variables are delocalized GICs or symmetry-adapted Cartesian displacements.

Older refinement workflows usually automate the solution of a least-squares problem after the user has chosen the coordinate model and constraints. `MORPHEUS` also automates the formulation of that problem. It analyzes the available isotopologues, detects weakly determined directions, preserves the maximum compatible symmetry and can propose explicit, rank-consistent constraints or shared parameter classes supported by the data.

2 Theoretical Background

`MORPHEUS` solves a constrained nonlinear least-squares problem in which the unknowns are not arbitrary Cartesian components but an algorithmically generated active structural model. At each geometry x, the calculated rotational observables are obtained from the principal moments of inertia,

$$I_\alpha(x) = \sum_i m_i \left(|\mathbf{r}_i|^2 - r_{i\alpha}^2 \right),$$

with isotope masses changed according to each isotopologue. The recommended fit observable is the corrected moment of inertia rather than the rotational constant itself, because it avoids the reciprocal transformation $B_\alpha = h/(8\pi^2 c I_\alpha)$ inside the weighted residual.

For an observed rotational constant B_α^0, the target equilibrium quantity is formed by applying vibrational and electronic corrections with the convention stored in `xyzin`:

$$B_\alpha^e \simeq B_\alpha^0 - \Delta B_\alpha^{\mathrm{vib}} - \Delta B_\alpha^{\mathrm{el}}.$$

The corresponding target moments define the residual vector $r(x)$. The least-squares objective is

$$\chi^2(x) = r(x)^T W r(x) + c(x)^T W_c c(x),$$

where W contains spectroscopic weights and $c(x)$ contains optional hard or soft structural information. Hard constraints are enforced by projection or large penalty weights; soft priors enter as additional weighted residuals.

The active displacement is written as

$$\Delta x = P\, \Delta y,$$

where P is the projector from active model variables to Cartesian displacements. For the GIC model, P is obtained from the frozen non-redundant, symmetry-labelled GIC B matrix supplied by `GICForge`. For the symmetry-adapted Cartesian model, P is the translationally and rotationally cleaned Cartesian symmetry basis. In both cases, only the totally symmetric subspace is used for a symmetry-preserving equilibrium refinement.

Primitive constraints, predicates and parameter classes are not separate coordinate systems. They are primitive structural relations that are projected onto the active model. If $s(x)$ is a primitive relation, its linearized row in active variables is

$$\frac{\partial s}{\partial y} = \frac{\partial s}{\partial x} P.$$

This is the key design rule: users, advisors and imported literature data can refer to chemically transparent distances, angles, dihedrals and out-of-plane coordinates, while the solver operates in a rank-consistent active space.

The numerical solver uses weighted normal equations with rank diagnostics, line-search safeguards and topology checks. The topology perceived at the initial geometry is frozen during an ordinary fit. If a step would create spurious contacts or change the covalent graph, the step is rejected rather than allowing the model itself to change during refinement.

3 Installation and Environment

The standard project setup is:

```
matrix-set
matrix-run
```

The setup script prepares the Python environment used by the GUI and command line, including RDKit and PySide6 when available. The Fortran backends are kept as independent executables and are selected explicitly from the GUI or with the `-backend` option. Check the resolved configuration with:

```
python -m matrix config
python -m matrix backends
```

The current production backend for MORPHEUS/SEfit is Python. The Fortran77 backend is maintained where required for common MATRIX services, especially `GICForge` consistency, but the Python MORPHEUS implementation is the reference path for the full adaptive least-squares solver and reports.

4 Overall Workflow

A reproducible `MORPHEUS` calculation has five stages.

1. Prepare a parent Cartesian geometry and isotopologue data.
2. Convert external sources into the common `xyzin` molecular container.
3. Validate the canonical input and inspect the generated diagnostics.
4. Inspect active coordinates, constraints, ranks, weights and warnings.

5. Run the fit and inspect residuals, uncertainties, correlations and the optimized-space Hessian.

External files are import sources only. Once a `xyzin` file exists, all `ORACLE` modules should read geometry, symmetry, rotational, vibrational and isotopologue information from that single container. `MORPHEUS` can update the container during preprocessing, but the fit itself rereads the canonical data.

5 Coordinate Models

5.1 GIC model

The default active model is:

```
coordinate_model = "gic"
```

`GICForge` reads atomic numbers and Cartesian coordinates, builds primitive internal coordinates, reduces them to a non-redundant generalized internal coordinate (GIC) set, assigns symmetry labels and provides the active coordinates and B matrix. The primitive classes include bond stretches, valence bends, dihedrals, out-of-plane coordinates, linear bends and ring coordinates. Ring coordinates are based on endocyclic dihedrals and analogous valence-angle combinations rather than Cremer–Pople variables.

Only totally symmetric coordinates are optimized in a symmetry-preserving refinement. The Gaussian-style input writer can therefore place the totally symmetric GICs first and all other symmetry species after the blank separator when an external geometry optimization should preserve symmetry.

5.2 Symmetry-adapted Cartesian model

The alternative active model is:

```
coordinate_model = "cartesian_symmetry"
```

This model removes translations and rotations from Cartesian displacements and symmetry-adapts the remaining space. Only totally symmetric Cartesian directions are fitted. This option does not require an active GIC B matrix, but constraints are still defined in primitive internal coordinates and are projected onto the Cartesian active space.

5.3 Choosing a model

Use the GIC model for production structural refinements when chemically local constraints, parameter classes and primitive uncertainties are central to the interpretation. Use the symmetry-adapted Cartesian model as an independent coordinate check or when a fully democratic Cartesian active space is preferred. Both models return final Cartesian coordinates, from which all primitive geometrical parameters and their uncertainties are propagated.

6 Input Files

6.1 Canonical `xyzin` container

`xyzin` is the common `ORACLE` molecular container. It starts with a normal XYZ block and then stores optional uppercase sections. The section used by `MORPHEUS` is `#ISOTOPOLOGUES`.

```
#ISOTOPOLOGUES
SCHEMA oracle.xyzin.isotopologues.v1
UNITS ROTATIONAL=MHz DELTAVIB=MHz DELTAEL=MHz SIGMA=MHz
INDEXING ATOMS=ONE_BASED
BEGIN parent
DEFINITION parent
ROTATIONAL_MHZ A=1000.000 B=800.000 C=600.000
DELTAVIB_MHZ A=1.0 B=2.0 C=3.0 SOURCE=qm CONVENTION=subtract
DELTAEL_MHZ A=0.0 B=0.0 C=0.0 SOURCE=none CONVENTION=subtract
SIGMA_MHZ A=0.01 B=0.01 C=0.01
END
BEGIN C1_13
DEFINITION 1:13
ROTATIONAL_MHZ A=990.000 B=790.000 C=590.000
END
```

`DEFINITION` uses one-based atom indexes. Multiple substitutions are separated by semicolons, for example `2:2;5:13`. `parent` means no isotopic substitution. `SIGMA_MHZ=inf` is accepted for a component with zero least-squares weight; each isotopologue must still retain at least one positive finite component weight.

A complete standalone `xyzin` input can also contain the `#MORPHEUS` run section and be passed directly to the command line:

```
python -m matrix semiexp --xyzin molecule.xyzin --outdir run
```

In that mode the initial Cartesian XYZ block and `#ISOTOPOLOGUES` provide the molecular and spectroscopic input. The `#MORPHEUS` section selects the active coordinate model and optional predicates:

```
#MORPHEUS
COORDINATE_MODEL = gic
OBSERVABLE = moments
COMPONENTS = Ia,Ib,Ic
FIT_COORDINATES = gic
FIXED_PARAMETER = @hydrogen_parameters
PREDICATES = INITIAL_GEOMETRY REFERENCE_LEVEL=medium DISTANCE_SIGMA=AUTO ANGLE_SIGMA=
    AUTO DIHEDRAL_SIGMA=AUTO
PREDICATE_SCOPE = xy_bonds,xh_bonds,heavy_angles,coh_angles,ring_torsions
PRIMITIVE_CLASS_ADVISOR = AUTO_SYNTHON INCLUDE=bonds,angles MIN_GROUP_SIZE=2
SYNTHON_LEVEL = auto
PRIMITIVE_CLASS_MIN = 0.70
PRIMITIVE_CLASS_CROSS_MAX = 0.20
PRIMITIVE_CLASS_BUDGET = auto
```

`COORDINATE_MODEL` and `FIT_COORDINATES` accept `gic` or `cartesian_symmetry`. `COMPONENTS` may be written as `ABC`, `AB`, `AC`, `BC` or in moment notation such as `Ia,Ib,Ic`. `PREDICATES=INITIAL_GEOMETRY` generates weighted primitive predicates centered on the initial Cartesian geometry using the requested scope and sigmas. If a sigma is `AUTO`, MORPHEUS selects it from `REFERENCE_LEVEL`: `high`, `medium`, `low`, or `kraitchman`. These are input priors, not output diagnostics.

`xy_bonds` denotes bonds between non-hydrogen atoms, while `xh_bonds` denotes any X–H bond; the legacy `heavy_bonds` and `oh_bonds` scopes remain accepted but new inputs should use the general XY/XH names. `PRIMITIVE_CLASS_ADVISOR=AUTO_SYNTHON` builds bond and angle primitive classes from MATRIX topology and atomic synthons. The synthon descriptors are continuous; `SYNTHON_LEVEL=coarse`, `medium`, or `fine` changes the quantization thresholds for co-

valency, delocalization, strain, and bond order. With `SYNTHON_LEVEL=auto`, MORPHEUS runs a short internal scan over `coarse`, `medium`, and `fine`; the candidate results are written under `_synthon_auto_candidates`. The selected level is the simplest model that satisfies rank and propagated-uncertainty checks for XY bonds, X–H bonds, and heavy-atom angles. Non-CH X–H bonds use the same threshold as XY bonds; C–H bonds use a wider threshold because C–D substitutions are often absent and C–H distances are therefore less directly determined. The advisor maps the resulting primitive classes onto the actual reduced GICs, freezes unsupported directions, limits the number of active classes to the experimental data budget when `PRIMITIVE_CLASS_BUDGET=auto`, and prints explicit accept/reject decision lines. Explicit user classes remain available through `PRIMITIVE_CLASS = name:R(1,2)|R(2,3)` entries.

Selector quantity	Meaning	Default acceptance threshold
XY distance	bond between two non-H atoms	0.002 Å
non-CH X–H distance	X–H bond with X different from C	0.002 Å
C–H distance	C–H bond	0.005 Å
heavy-atom angle	valence angle without H atoms	0.2°

The broader C–H threshold is not a special rule for sugars. It reflects the general information content of many rotational datasets: heavy-atom isotopic substitution constrains the heavy skeleton much more directly than C–H distances, and deuterated isotopologues are often absent. Non-CH X–H bonds are kept with the tighter XY threshold unless the user explicitly chooses a different predicate or class policy.

Validate a container with:

```
python -m matrix xyzin-validate --xyzin working/xyzin
python -m matrix xyzin-validate --xyzin working/xyzin --require-semiexp-observations
```

6.2 Standalone job file

The recommended standalone input extension is `.mse.toml`. The schema identifier is:

```
oracle.semiexp.job.v1
```

```
schema = "oracle.semiexp.job.v1"
title = "example MORPHEUS fit"

[fit]
backend = "python"
coordinate_model = "gic"
observable = "moments"
rotational_components = "auto"
robust_loss = "none"

[geometry]
units = "angstrom"
atoms = [
  ["O",  0.000000,  0.000000,  0.000000],
  ["H",  0.000000,  0.000000,  0.960000],
  ["H",  0.920000,  0.000000, -0.240000],
]
```

```
[[isotopologues]]
label = "parent"
[isotopologues.definition]
substitutions = ""
[isotopologues.constants]
A_MHz = 1000.0
B_MHz = 800.0
C_MHz = 600.0
```

Inline `[[isotopologues]]` records can be replaced by a separate observations file referenced from a `[files]` table. Relative paths are resolved from the job-file directory.

6.3 Legacy MSR import

Legacy MSR files are accepted for compatibility with explicit extensions: `.msr`, `.msr.inp` and `*_msr.inp`. Generic `.inp` files are not auto-detected as MSR files.

The importer accepts Z-matrix or Cartesian parent geometries, isotope mass blocks, `bexp`, `dbvib`, `dbele`/`dbelec`, `weights`, frozen Z-matrix variables marked by `#`, and `PREDICATES` records. The imported geometry is converted to Cartesians. Frozen Z-matrix variables are translated to primitive constraints when they refer to real atoms. Legacy `constraints:` records are preserved as diagnostic compatibility metadata unless they are expressed in the current MORPHEUS constraint language.

7 Constraints, Predicates and Parameter Classes

7.1 Hard constraints

Hard constraints remove directions from the least-squares space. They can be specified as primitive freezes, Gaussian ModRedundant lines or Gaussian-style GIC expressions.

```
R(1,2) Frozen
A 2 1 3 F
DRCH(Frozen,Value=0.0)=R(5,8)-R(5,7)
ROH(Frozen,Value=0.965159)=R(6,7)
```

The `Value=` keyword follows Gaussian convention and imposes an explicit target. Without `Value=`, the initial value is frozen. Constraints are expanded over symmetry-equivalent primitive parameters when required.

7.2 Predicates

Predicates are weighted structural observations rather than exact constraints. They are useful when accurate electronic-structure or library information should stabilize an underdetermined refinement without being treated as exact.

```
python -m matrix semiexp --job input.mse.toml --outdir run \
  --qm-predicate "R(2,3):0.9683:0.002:reference"
```

The format is `label_pattern:value:sigma:source`. Smaller `sigma` values make the predicate stronger.

7.3 Parameter classes

Parameter classes tie several active corrections together. A shared class does not force final primitive values to be identical; it imposes a common correction in the active least-squares space. This is appropriate when related parameters are not independently determined by the available isotopologues.

```
python -m matrix semiexp --job input.mse.toml --outdir run \
  --parameter-class "CH_stretches:shared:R(1,6)|R(2,7)|R(3,8)"
```

Automatic class suggestions are conservative. They are generated from synthon descriptors and then validated in the active totally symmetric GIC space. A primitive pattern is accepted only when it is a dominant contribution to the matched GIC; weak appearances inside highly delocalized GICs are ignored. This prevents a class from accidentally capturing unrelated totally symmetric directions.

7.4 Model-layer-generated constraints

The advisor analyzes symmetry, isotopic coverage and reference structural information. It is a conservative tuning layer on top of a chemically valid model, not an automatic replacement for that model. A run requested with `-apply-sensitivity-advisor` is accepted only if the base model already passes the MORPHEUS rank and validation checks and the advisor candidate does not worsen the rotational residuals, rank, conditioning or fitted geometry beyond the configured gate thresholds. If the base model is underdetermined, the advisor report is still written for diagnosis, but its suggestions are not applied automatically; the user must first add justified predicates, parameter classes or fixed coordinates. Typical actions are:

- freeze hydrogen local geometry when corresponding deuterations are absent;
- preserve near-C_s or higher symmetry by constraining coordinates that should vanish in the higher-symmetry limit;
- generate shared classes for chemically equivalent corrections based on synthon descriptors;
- report weak or poorly observed directions before the fit.

```
python -m matrix semiexp --job input.mse.toml --outdir run \
  --fix-hydrogens \
  --qm-predicate "R(2,3):0.9683:0.002:reference" \
  --parameter-class "CH_stretches:shared:R(1,6)|R(2,7)|R(3,8)"
```

8 Least-Squares Solver

`MORPHEUS` uses an adaptive Levenberg–Marquardt trust-region solver with rank-revealing SVD steps, dynamic column scaling, optional robust losses, line-search safeguards, checkpoint/restart support and deterministic conditioning diagnostics. The GIC coordinate definition is built once for a fresh fit and then reused; the B matrix is recomputed or updated when needed. Restart jobs rebuild the GIC definition deterministically from the restart geometry.

Option	Meaning
`backend`	Requested numerical backend: `python` or `fortran77`.
`coordinate_model`	`gic` or `cartesian_symmetry`.
`observable`	`moments`, `rotational_constants` or `auto`. `moments` is recommended.
`rotational_components`	`auto`, `ABC`, `AB`, `AC` or `BC`. Planar molecules use only two independent components.
`max_iter`	Explicit iteration cap. If omitted, the cap is proportional to the number of effective parameters.
`damping`	Initial Levenberg–Marquardt damping.
`max_step`	Maximum active-coordinate trust-region step norm.
`prune_condition`	Optional deterministic weak-parameter pruning target. `0.0` disables pruning.
`robust_loss`	`none`, `huber`, `soft_l1` or `cauchy`.
`robust_scale`	Robust residual scale in weighted units; `0` selects an automatic median-absolute-deviation scale.
`leave_one_out`	Runs exact leave-one-isotopologue-out refits after the final fit.
`checkpoint/restart`	Save or restart from a checkpoint JSON.

9 Planar Molecules

A planar molecule has only two independent rotational constants. When `rotational_components = "auto"`, `MORPHEUS` chooses among `AB`, `AC` and `BC`. In moment mode these are mapped to (I_a, I_b), (I_a, I_c) and (I_b, I_c). The automatic choice favors the most stable pair according to the propagated instability of the omitted component and then uses Jacobian conditioning as a tie-breaker. The omitted rotational component is still calculated and reported as a diagnostic.

10 Command-Line Workflows

10.1 Input validation

```
python -m matrix xyzin-validate --xyzin working/xyzin
python -m matrix xyzin-validate --xyzin working/xyzin \
  --require-semiexp-observations
```

The validation commands check the canonical MATRIX container before a production fit. A full MORPHEUS run then materializes or updates `xyzin`, constructs the coordinate model, applies requested constraints/classes and writes rank, conditioning, planar-component choice and warning diagnostics to the output manifest.

10.2 Production fit

```
python -m matrix semiexp \
  --job input.mse.toml \
  --xyzin working/xyzin \
  --outdir working/morpheus/run01 \
```

```
  --backend python \
  --coordinate-model gic \
  --observable moments \
  --rotational-components auto
```

10.3 Class comparison

To test whether shared classes are justified, run the same input twice: once without the class and once with the proposed shared or fixed class.

```
python -m matrix semiexp --job p-EBN.msr.inp --outdir run_no_class
python -m matrix semiexp --job p-EBN.msr.inp --outdir run_classed \
  --parameter-class "example:shared:R(1,2)|R(3,4)"
```

The comparison should be discussed in terms of rank, residuals and condition number. Class-tied uncertainties are conditional on the shared/fixed class model and should not be used as independent evidence for every primitive parameter.

10.4 Explaining GIC definitions

For debugging or method reporting, `GICForge` definitions can be exported with dominant primitive contributions:

```
python -m matrix gic-define --geometry parent.xyz \
  --out gic_definition.json \
  --explain-out gic_explain.json \
  --explain-csv gic_explain.csv
```

The explanation lists the final GIC labels, symmetry irreps, coordinate kinds, atom indexes, number of contributing primitives and the largest primitive weight fractions.

11 Graphical Interface

The GUI uses the same services as the command line. A typical GUI run is:

1. create or load a MATRIX project;
2. import Cartesian geometry or a legacy/job input;
3. inspect or edit isotopologue definitions, rotational constants, vibrational corrections, electronic corrections and standard deviations;
4. choose Python or Fortran77 backend for each computational step;
5. choose GIC or symmetry-Cartesian active coordinates;
6. validate the input and inspect warnings;
7. open the semiexperimental model tab and accept, edit or reject model-layer-generated constraints/classes with their reason, confidence, atom list and evidence source visible;
8. run the fit and inspect output tables.

The GUI must not keep a private isotopologue store. Imports from tables, MSR files or other sources are written into `xyzin` and then reread by the MORPHEUS service.

12 Output Files

Each run writes a text report, CSV diagnostics and a manifest. The standard output directory contains:

File	Content
`semiexp_report.txt`	Human-readable summary of method, constraints, diagnostics, residuals and final geometry.
`semiexp_report.json`	Versioned machine-readable report with method, model, provenance, diagnostics, warnings, residuals and final geometry.
`semiexp_geometry.xyz`	Optimized Cartesian structure.
`semiexp_geometry_parameters.csv`	Primitive bond lengths, angles, dihedrals and propagated standard errors.
`semiexp_rotational_constants.csv`	Experimental corrected, calculated and residual rotational constants.
`semiexp_residuals.csv`	Least-squares residual rows.
`semiexp_parameters.csv`	Active coordinate values, uncertainties and class labels.
`semiexp_covariance.csv` and `semiexp_correlation.csv`	Statistical covariance and correlation matrices.
`semiexp_svd_diagnostics.csv`	Singular values, numerical rank and conditioning diagnostics.
`semiexp_uncertainty_diagnostics.csv`	Warnings for unusually large or poorly determined uncertainties.
`semiexp_warnings.csv`	Machine-readable warnings.
`semiexp_hessian.csv` and `semiexp_hessian_eigenvalues.csv`	Optimized-space Hessian and stationary-point check.
`semiexp_kraitchman.csv`	Kraitchman diagnostic values when applicable.
`semiexp_checkpoint.json`	Restart coordinates and metadata.
`semiexp_manifest.json`	Input, option and output provenance.

The final report gives Cartesian coordinates and conventional primitive coordinates in Å and degrees. Their uncertainties are obtained by propagating the fitted covariance through the final Cartesian geometry.

13 Diagnostics and Troubleshooting

Symptom or warning	Interpretation and action
`rank reduced`	The experimental data or constraints do not determine all active directions. Inspect singular vectors, add justified predicates, classes or constraints, or reduce the active space.

Symptom or warning	Interpretation and action
`large condition number`	The fit is numerically sensitive. Try moment observables, automatic planar-component selection, shared classes or better experimental uncertainties.
`large parameter uncertainty`	The parameter is weakly observed. Check isotopic coverage and whether a chemically justified class or predicate is needed.
`low robust weight`	An isotopologue behaves as an outlier under the selected robust loss. Verify its assignment, constants and corrections.
`planar pair warning`	The selected two-component pair is poorly conditioned. Use `rotational_components=auto` unless there is a specific spectroscopic reason not to.
`not a minimum`	The optimized-space Hessian has negative curvature. Check constraints, symmetry, data consistency and the reference geometry.

14 Regression Tests

The main scientific contract tests are:

```
python -m pytest gui/tests/test_scientific_contracts.py -q
```

They cover `xyzin` validation, legacy import, GIC behavior, constraints, predicates, parameter classes, planar-component selection, Kraitchman diagnostics, uncertainty propagation and representative benchmark fits. The parameter-class regression verifies that automatic suggestions are filtered in the active totally symmetric GIC space and do not remove unrelated directions.

15 Recommended Practice

1. Use Cartesian geometries as the primary structure input.
2. Materialize all external data into `xyzin` before production fitting.
3. Use moments of inertia and `rotational_components=auto`.
4. Validate `xyzin` and input observations before a fit.
5. Treat generated constraints and classes as documented model choices: inspect them, keep their provenance, and report them.
6. For incomplete isotopic coverage, prefer chemically justified predicates or shared classes over arbitrary parameter deletion.
7. Check final rotational residuals, propagated primitive uncertainties, high correlations, leave-one-out diagnostics and Hessian eigenvalues.
8. Keep the output manifest with any published or archived result.

16 Compatibility Notes

Legacy MSR inputs are supported to recover existing data sets and to compare results. They are not used as active Z-matrix models. `MORPHEUS` converts the imported information into Cartesians, isotopologue records, primitive constraints, predicates and optional classes, then rebuilds the active model from `GICForge` or symmetry-adapted Cartesians. The goal is not to obtain a different structure, but to obtain the same physical structure from a unique, topology-derived, symmetry-consistent and statistically documented model.